SMG · CYBERSECURITY

Relatório de Auditoria de Segurança

SOU MANA.GER — SMG Barber (sou-manager)

Data da Auditoria: 03/11/2026

Ambiente: Produção (Vercel) + Supabase PostgreSQL

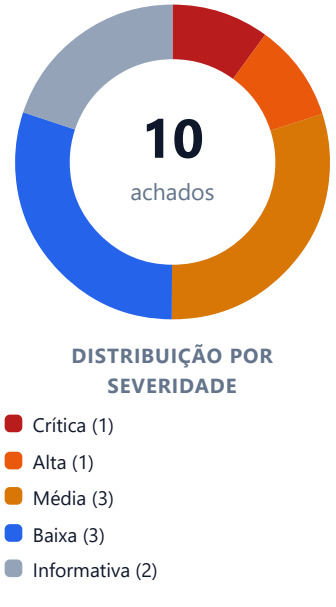
Escopo Auditado: Banco de Dados (RLS/RPC), Edge Functions, Frontend, Segredos e Histórico Git

Total de Achados: 10 (1 crítico · 1 alto · 3 médios · 3 baixos · 2 informativos)

Nota Metodológica: A auditoria cobriu as cinco categorias essenciais mapeadas para a stack real do projeto (React 19 + TypeScript + Supabase + PostgreSQL 16 + Deno Edge Functions):
(1) **Banco sem tranca** → Mapeado para Row Level Security (RLS) no PostgreSQL e checagem de tenant_id via Supabase Auth;
(2) **Permissão no navegador** → Cruzamento entre gates de UI (ManagerRoute / SuperAdminRoute) e validações no servidor (RPCs, Edge Functions e API REST);
(3) **IDOR** → Verificação sistemática de endpoints e funções PL/pgSQL que recebem identificadores de objetos para assegurar validação de posse;
(4) **Chaves expostas** → Varredura de código-fonte, bundles de frontend compilados, histórico Git e arquivos locais desprotegidos;
(5) **Inputs sem tratamento (XSS)** → Análise de sinks no DOM (document.write, innerHTML) e fluxos de impressão/exportação.

1. Resumo Executivo

1	1	3	3
Crítica	Alta	Média	Baixa / Info 2





2. Pontos Fortes e Riscos Centrais

Pontos Fortes Verificados no Código:

- RLS ativa em 100% das tabelas do banco de dados (70+ tabelas verificadas nas 159 migrations; nenhuma tabela com RLS desligada).
- Isolamento multi-tenant canônico rigorosamente aplicado via current_tenant_id_from_auth_uid() em mais de 240 políticas de segurança.

- Correção crítica de bulk_close_comandas_admin (20260831120000): exige auth.uid(), valida permissão de gerente (owner/admin/manager), rejeita tenant nulo e valida a posse de cada comanda individualmente no lote.
- Guarda rígida em batch_unblock_comandas (20260910160000): valida auth.uid(), bloqueia cruzamento de tenants (cross-tenant check) e exige papel gerencial.
- Hardening de approve_access_request (20260914120000): corpo da RPC em SECURITY DEFINER valida explicitamente auth.uid() e exige superadmin (current_is_super_admin_from_auth_uid).
- Proteção em provision_new_tenant (20260805000000): bloqueia auth.uid() nulo e assegura que p_user_id coincida com o usuário autenticado.
- Desativação completa da RPC legada close_order (20260813130200): revogada para anon, authenticated e PUBLIC, restrita exclusivamente à service_role.
- Hierarquia estrita de criação de usuários em admin-create-user (linhas 135-141): função canAssignStaffRole() bloqueia manager de conceder privilégio admin.
- Autenticação e autorização em supabase-usage-monitor (linhas 62-75): exige cabeçalho Authorization Bearer com JWT válido e valida papel de superadmin antes de aceitar ingestão de métricas.
- Assinatura criptográfica segura em site-sanchez-appointments (linhas 59-101): validação de webhook via HMAC-SHA256 comparada através de timingSafeEqual (proteção contra timing attacks).
- Propagação de JWT em invite-team-member (linhas 41-48): a Edge Function delega a autorização e o papel para a RPC invite_team_member com o token do chamador.
- Tratamento preventivo de XSS em fluxos de impressão: CashClosingPage.tsx:260-264 e Comandas.tsx:1247-1252 utilizam escapeHtml() e anulam o opener (printWindow.opener = null). Testes automatizados em tests/security/f5_1_f5_2_xss_print_escape.test.ts.
- Sanitização estrita de URLs em agendamento público: funções buildSafeInstagramUrl, buildSafeWhatsAppUrl e buildSafeHttpsUrl (BookingHeader.tsx / PublicBookingPage.tsx) rejeitam esquemas perigosos (javascript:).
- Ausência total de sinks perigosos de injeção no frontend: nenhum uso de dangerouslySetInnerHTML, eval() ou new Function() em toda a árvore de código da aplicação.

Pontos Fracos e Riscos Centrais:

- **Exposição de segredos de produção no histórico Git:** O commit 46e21ce incluiu a chave `service_role` e credenciais de banco, exigindo rotação completa de segredos.
- **Chave de IA exposta no bundle do cliente:** A `VITE_GEMINI_API_KEY` é embutida estaticamente no JavaScript público entregue ao navegador.
- **Divergência de privilégios entre UI e REST:** Usuários operacionais sem permissão visual conseguem alterar serviços/preços se chamarem a API REST do Supabase diretamente.
- **Higiene de arquivos locais:** Arquivos de segredos como `secrets.env.txt` não estão cobertos pelo `.gitignore`.

3. Tabela de Achados Detalhados

Severidade	Arquivo e Linha	Descrição
CRITICA	`git commit 46e21ce (.vercel-temp.env no histórico do repositório)` git show 46e21ce:.vercel-temp.env (SUPABASE_SERVICE_ROLE_KEY, SUPABASE_JWT_SECRET, POSTGRES_PASSWORD, VERCEL_OIDC_TOKEN)	**F4.1** — Exposição de segredos de produção no histórico Git: o arquivo .vercel-temp.env foi commitado com credenciais administrativas completas. Embora tenha sido desindexado no commit c1711da e adicionado ao .gitignore, qualquer clone/fork mantém acesso aos segredos em texto plano.
ALTA	`vite.config.ts / services/geminiService.ts` vite.config.ts:18-20 ("process.env.GEMINI_API_KEY": JSON.stringify(env.VITE_GEMINI_API_KEY)) e geminiService.ts:3-10	**F4.2** — Chave de API do Google Gemini injetada diretamente no bundle estático do frontend entregue ao navegador do cliente.

MEDIA	`secrets.env.txt / env.local.bak_bom.txt (raiz do projeto)` secrets.env.txt:1-12 e env.local.bak_bom.txt:1-15 (arquivos locais não cobertos pelo .gitignore)	**F4.3** — Arquivos contendo segredos reais e backups de configuração na raiz do repositório desprotegidos contra comandos "git add .".
INFORMATIVA	`src/lib/supabase/client.ts` 18-21 (LOCAL_DEMO_EMAIL = "teste@soumanager.local", LOCAL_DEMO_PASSWORD = "12345678")	**F4.4** — Credenciais de teste locais embutidas no código-fonte.
MEDIA	`supabase/migrations/20260212000000_initial_schema.sql (políticas RLS de tabelas mestres)` Ex.: policies em services, products, categories (USING / WITH CHECK tenant_id = current_tenant_id_from_auth_uid())	**F2.1** — Mutações diretas via Supabase REST API em tabelas de catálogo/configuração permitidas a qualquer funcionário autenticado do mesmo tenant.
MEDIA	`supabase/migrations/20260418100000_add_service_execution_participants.sql` 109: GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE public.service_execution_participants TO anon;	**F1.1** — Concessão excessiva de permissões a nível de tabela para o papel "anon".
BAIXA	`supabase/migrations/20260807000000_phase_6_0_5_3_feature_flags.sql` 80: GRANT INSERT, UPDATE, DELETE ON TABLE public.feature_flags TO authenticated;	**F1.2** — Grant DML irrestrito a todos os usuários autenticados em tabela protegida por policy exclusiva de superadmin.
INFORMATIVA	`supabase/migrations/20260806090000_phase_6_0_5_2_plans_catalog.sql` Catálogos públicos (plans, features, plan_features): USING(true) + GRANT SELECT TO authenticated	**F1.3** — Tabelas de catálogo de planos com leitura global intencional.
BAIXA	`supabase/migrations/20260212000000_initial_schema.sql (funções financeiras pay_account_payable)` RPC pay_account_payable / cancel_account_payable (validação restrita a tenant_id)	**F3.1** — RPCs financeiras pontuais que recebem ID de título a pagar e não validam papel gerencial do autorizador.
BAIXA	`pages/CashClosingPage.tsx:261 e pages/Comandas.tsx:1249` CashClosingPage.tsx:261 e Comandas.tsx:1249 (printWindow.document.write)	**F5.1** — Uso residual do método legado document.write para montagem de layouts de impressão em janela about:blank.

4. Detalhamento dos Achados

CRITICA 4. Chaves expostas · F4.1 `git commit 46e21ce (.vercel-temp.env no histórico do repositório)`

Exposição de segredos de produção no histórico Git: o arquivo .vercel-temp.env foi commitado com credenciais administrativas completas. Embora tenha sido desindexado no commit c1711da e adicionado ao .gitignore, qualquer clone/fork mantém acesso aos segredos em texto plano.

Localização: git show 46e21ce:.vercel-temp.env (SUPABASE_SERVICE_ROLE_KEY, SUPABASE_JWT_SECRET, POSTGRES_PASSWORD, VERCEL_OIDC_TOKEN)

Por que é explorável: A chave service_role e o POSTGRES_PASSWORD concedem bypass irrestrito de RLS e acesso direto ao banco PostgreSQL. Se as chaves não foram revogadas e rotacionadas nas plataformas Supabase e Vercel, o banco de dados de produção permanece comprometido.

Impacto: Comprometimento total da confidencialidade e integridade dos dados multi-tenant, bypass de RLS e risco na cadeia de deploy.

ALTA 4. Chaves expostas · F4.2 `vite.config.ts / services/geminiService.ts`

Chave de API do Google Gemini injetada diretamente no bundle estático do frontend entregue ao navegador do cliente.

Localização: vite.config.ts:18-20 ("process.env.GEMINI_API_KEY": JSON.stringify(env.VITE_GEMINI_API_KEY)) e geminiService.ts:3-10

Por que é explorável: Qualquer usuário ou agente malicioso inspecionando o código-fonte JS ou o tráfego de rede no navegador pode extrair a API Key e utilizá-la em scripts externos, gerando exaustão de cotas de IA ou custos na conta de faturamento do Google Cloud.

Impacto: Esgotamento de cota da API de IA, negação de serviço funcional no assistente inteligente e custos financeiros imprevistos.

MEDIA 4. Chaves expostas · F4.3 `secrets.env.txt / env.local.bak_bom.txt (raiz do projeto)`

Arquivos contendo segredos reais e backups de configuração na raiz do repositório desprotegidos contra comandos "git add .".

Localização: secrets.env.txt:1-12 e env.local.bak_bom.txt:1-15 (arquivos locais não cobertos pelo .gitignore)

Por que é explorável: O arquivo .gitignore possui regras para "*.bak_bom", mas não para "*.bak_bom.txt" nem para "secrets.env.txt". O comando git check-ignore confirma que esses arquivos estão desprotegidos.

Impacto: Risco iminente de novo vazamento de segredos em commits rotineiros de desenvolvedores.

INFORMATIVA 4. Chaves expostas · F4.4 `src/lib/supabase/client.ts`

Credenciais de teste locais embutidas no código-fonte.

Localização: 18-21 (LOCAL_DEMO_EMAIL = "teste@soumanager.local", LOCAL_DEMO_PASSWORD = "12345678")

Por que é explorável: Ativadas somente quando NÃO existem variáveis de ambiente Supabase configuradas e o host é localhost/127.0.0.1. Não explorável em produção, mas a string de senha consta no bundle compilado.

Impacto: Higiene de código; baixo risco em ambiente de produção.

MEDIA 2. Permissão navegador · F2.1

`supabase/migrations/20260212000000_initial_schema.sql (políticas RLS de tabelas mestres)`

Mutações diretas via Supabase REST API em tabelas de catálogo/configuração permitidas a qualquer funcionário autenticado do mesmo tenant.

Localização: Ex.: policies em services, products, categories (USING / WITH CHECK tenant_id = current_tenant_id_from_auth_uid())

Por que é explorável: O frontend oculta a UI de configurações de catálogo para barbeiros e recepcionistas via ManagerRoute/SuperAdminRoute. Contudo, as políticas de RLS no PostgreSQL verificam apenas o tenant_id, permitindo que um usuário não-gerente envie requisições HTTP REST diretas (PATCH/DELETE) para alterar preços ou excluir serviços.

Impacto: Quebra de controle de acesso intra-tenant (um profissional pode alterar preços ou deletar produtos via cURL/Postman).

MEDIA 1. Banco sem tranca · F1.1 `supabase/migrations/20260418100000_add_service_execution_participants.sql`

Concessão excessiva de permissões a nível de tabela para o papel "anon".

Localização: 109: GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE public.service_execution_participants TO anon;

Por que é explorável: A migration 20260715010000 revogou apenas o INSERT para anon. Como a RLS está ativa e não há policies concedendo acesso para anon, o PostgreSQL bloqueia o acesso na prática. No entanto, o GRANT base viola a defesa em profundidade e o princípio do menor privilégio.

Impacto: Risco de exposição acidental caso uma política de RLS permissiva venha a ser criada no futuro.

BAIXA 1. Banco sem tranca · F1.2 `supabase/migrations/20260807000000_phase_6_0_5_3_feature_flags.sql`

Grant DML irrestrito a todos os usuários autenticados em tabela protegida por policy exclusiva de superadmin.

Localização: 80: GRANT INSERT, UPDATE, DELETE ON TABLE public.feature_flags TO authenticated;

Por que é explorável: A tabela possui RLS ativa com acesso restrito a superadmin, mas o grant na tabela sugere permissões inexistentes, violando a integridade das auditorias automatizadas de permissão de banco.

Impacto: Inconsistência de privilégios de banco de dados e violação de menor privilégio.

INFORMATIVA 1. Banco sem tranca · F1.3 `supabase/migrations/20260806090000_phase_6_0_5_2_plans_catalog.sql`

Tabelas de catálogo de planos com leitura global intencional.

Localização: Catálogos públicos (plans, features, plan_features): USING(true) + GRANT SELECT TO authenticated

Por que é explorável: Por design de negócio, planos e funcionalidades são de leitura pública entre os tenants. Toda escrita é restrita à service_role.

Impacto: Nenhum impacto de segurança (comportamento intencional e verificado).

BAIXA 3. IDOR · F3.1 `supabase/migrations/20260212000000_initial_schema.sql (funções financeiras pay_account_payable)`

RPCs financeiras pontuais que recebem ID de título a pagar e não validam papel gerencial do autorizador.

Localização: RPC pay_account_payable / cancel_account_payable (validação restrita a tenant_id)

Por que é explorável: Embora a função verifique que o título pertença ao tenant do chamador (evitando IDOR entre diferentes barbearias), qualquer funcionário autenticado do mesmo tenant que descubra o UUID pode solicitar a liquidação ou cancelamento.

Impacto: Operação financeira realizada por usuário sem atribuição formal de gestão de caixa dentro da mesma organização.

BAIXA 5. Inputs (XSS) · F5.1 `pages/CashClosingPage.tsx:261 e pages/Comandas.tsx:1249`

Uso residual do método legado document.write para montagem de layouts de impressão em janela about:blank.

Localização: CashClosingPage.tsx:261 e Comandas.tsx:1249 (printWindow.document.write)

Por que é explorável: Ambos os fluxos já aplicam escapeHtml() e desvinculam o opener (printWindow.opener = null), eliminando o risco ativo de XSS. No entanto, o uso de document.write é uma prática legada suscetível a erros futuros caso novos campos sejam incluídos sem o escape apropriado.

Impacto: Risco de manutenção futura; ausência de risco imediato no código atual.

5. Recomendações Priorizadas

P1 **Rotacionar credenciais vazadas no histórico Git e purgar histórico**
Rotacionar imediatamente no dashboard do Supabase e da Vercel: SUPABASE_SERVICE_ROLE_KEY, SUPABASE_JWT_SECRET, POSTGRES_PASSWORD e VERCEL_OIDC_TOKEN. Executar git filter-repo para expurgar o commit 46e21ce de forks e clones.

P1 **Isolar a chave Google Gemini através de Edge Function backend**
Remover VITE_GEMINI_API_KEY do vite.config.ts e criar a Edge Function supabase/functions/gemini-assistant. O frontend passa a chamar a Edge Function com o JWT autenticado do usuário, mantendo a API Key como secret seguro no backend.

P2 **Proteger arquivos de segredos locais no .gitignore**
Adicionar "secrets.env.txt", "*.bak_bom.txt" e "*.env.txt" no arquivo .gitignore para prevenir commits acidentais de credenciais da máquina de desenvolvimento.

P2 **Endurecer RLS em tabelas mestres contra mutações diretas por REST**

Atualizar as políticas de INSERT, UPDATE e DELETE de tabelas mestres (services, products, categories) para exigir papel de gerente através de "current_is_tenant_manager_from_auth_uid(tenant_id)".

P3 **Alinhar permissões residuais de tabela (revogar grants excessivos)**
Executar migration revogando "SELECT, UPDATE, DELETE ON service_execution_participants FROM anon" e "INSERT, UPDATE, DELETE ON feature_flags FROM authenticated".

P3 **Substituir document.write por estilos de impressão CSS (@media print)**
Refatorar os componentes de Fechamento de Caixa e Comandas para renderizar o layout de impressão diretamente no DOM via React com classes Tailwind @media print e window.print(), eliminando popups about:blank.

6. Issues para o GitHub

Os blocos abaixo contêm o conteúdo integral pronto para ser copiado e colado na criação de issues no GitHub.

ISSUE 1 — [Segurança] Segredos reais de produção expostos no histórico Git (.vercel-temp.env no commit 46e21ce)

```
--- ISSUE 1 ---
Título: [Segurança] Segredos reais de produção expostos no histórico Git (.vercel-temp.env no commit 46e21ce)
Labels: security, severidade-crítica

## Descrição do Problema
O arquivo `.vercel-temp.env` foi commitado no histórico do repositório no commit `46e21ce` contendo credenciais administrativas de produção em texto plano:
- `SUPABASE_SERVICE_ROLE_KEY`
- `SUPABASE_JWT_SECRET`
- `POSTGRES_PASSWORD`
- `VERCEL_OIDC_TOKEN`

Embora o commit `c1711da` tenha desindexado o arquivo do rastreamento ativo e adicionado regra ao `.gitignore`, os segredos permanecem intactos e acessíveis no histórico de qualquer clone ou fork do repositório.

## Por que é explorável
A chave `service_role` e as credenciais de banco de dados ignoram completamente as políticas de Row Level Security (RLS) do Supabase. Qualquer pessoa que clone o repositório ou acesse um fork tem capacidade de consultar, alterar ou excluir dados de todos os tenants em produção. O token OIDC da Vercel permite acionar ou modificar deploys na nuvem.

## Evidência
```bash
git show 46e21ce:.vercel-temp.env
# Exibe SUPABASE_SERVICE_ROLE_KEY, SUPABASE_JWT_SECRET, POSTGRES_PASSWORD e VERCEL_OIDC_TOKEN
```

Impacto
Comprometimento total da segurança, confidencialidade e integridade dos dados multi-tenant, além da cadeia de deploy de produção.

Sugestão de Correção
1. Acessar o painel do Supabase e rotacionar imediatamente a `service_role key`, o `JWT Secret` e alterar a `database password`.
2. Acessar o painel da Vercel e revogar o `VERCEL_OIDC_TOKEN`.
3. Executar o utilitário `git-filter-repo` ou `BFG Repo-Cleaner` para purgar o arquivo `.vercel-temp.env` do histórico Git do repositório remoto.

Critérios de Aceite
- [ ] Novas chaves `service_role` e `POSTGRES_PASSWORD` geradas no Supabase
- [ ] Token Vercel revogado e reconfigurado nas variáveis de CI/CD
- [ ] Execução de `git log -S "SUPABASE_SERVICE_ROLE_KEY"` não retorna o segredo antigo
- [ ] Auditoria de segurança de segredos (gitleaks) concluída sem alertas
--- FIM ISSUE 1 ---
```

ISSUE 2 — [Segurança] Chave Google Gemini exposta no bundle estático do frontend (VITE_GEMINI_API_KEY)

```
--- ISSUE 2 ---
Título: [Segurança] Chave Google Gemini exposta no bundle estático do frontend (VITE_GEMINI_API_KEY)
Labels: security, severidade-alta

## Descrição do Problema
No arquivo `vite.config.ts` (linhas 18-20), a variável de ambiente `VITE_GEMINI_API_KEY` é embutida globalmente no objeto
`process.env.GEMINI_API_KEY` e consumida diretamente pelo navegador em `services/geminiService.ts`:
```typescript
// vite.config.ts:18-20
'process.env.GEMINI_API_KEY': JSON.stringify(env.VITE_GEMINI_API_KEY || env.GEMINI_API_KEY || '')
```

Por que é explorável
O Vite substitui as referências a `process.env.GEMINI_API_KEY` pelo valor literal da string durante o build de produção. A chave
fica disponível em texto plano no arquivo `dist/assets/index-*.js`. Qualquer usuário da aplicação pode extrair a credencial da aba
Rede ou Console das Ferramentas de Desenvolvedor e utilizá-la em scripts próprios.

Evidência
- `vite.config.ts:18-20`
- `services/geminiService.ts:3-10`
- Presença da chave em chamadas diretas client-side para `https://generativelanguage.googleapis.com`.

Impacto
Esgotamento de cotas de IA, interrupção das funcionalidades de inteligência artificial para usuários legítimos e possíveis
cobranças não autorizadas na conta de faturamento do Google Cloud.

Sugestão de Correção
1. Criar uma Edge Function no Supabase (ex.: `supabase/functions/gemini-assistant`).
2. Configurar o secret `GEMINI_API_KEY` exclusivamente no ambiente do Supabase CLI / Edge Functions.
3. Fazer o frontend chamar a Edge Function enviando o Bearer Token JWT do usuário autenticado.
4. Remover a injeção da chave em `vite.config.ts`.

Critérios de Aceite
- [ ] Nenhuma chamada direta do navegador para a API do Google Gemini
- [ ] Chave `VITE_GEMINI_API_KEY` ausente do arquivo de bundle compilado (`dist/`)
- [ ] Edge Function valida o JWT do chamador antes de repassar a consulta ao modelo de IA
--- FIM ISSUE 2 ---
```

ISSUE 3 — [Segurança] Mutações diretas via Supabase REST API em tabelas mestres sem validação de papel gerencial

````
--- ISSUE 3 ---
Título: [Segurança] Mutações diretas via Supabase REST API em tabelas mestres sem validação de papel gerencial
Labels: security, severidade-média

## Descrição do Problema
Em tabelas como `services`, `products` e `categories` (criadas em `20260212000000_initial_schema.sql`), as políticas de RLS de escrita (`INSERT`, `UPDATE`, `DELETE`) validam apenas a cláusula:
```sql
USING (tenant_id = current_tenant_id_from_auth_uid())
WITH CHECK (tenant_id = current_tenant_id_from_auth_uid())
```

## Por que é explorável
O frontend esconde as telas de gestão de catálogo para usuários de cargos operacionais (`barber` e `receptionist`) via componentes de rota (`ManagerRoute`). No entanto, a API REST do Supabase não possui esse gate no banco de dados. Um barbeiro autenticado pode emitir requisições HTTP REST com seu token JWT para alterar o preço de serviços ou excluir itens do catálogo.

## Evidência
- Políticas de RLS em `services` e `products` checam apenas `tenant_id = current_tenant_id_from_auth_uid()`.
- Ausência da verificação `current_is_tenant_manager_from_auth_uid(tenant_id)` nas operações de escrita.

## Impacto
Violação de controle de acesso baseado em papel (RBAC), permitindo sabotagem operacional ou alteração indevida de valores financeiros de serviços.

## Sugestão de Correção
Adicionar migration atualizando as políticas de `INSERT`, `UPDATE` e `DELETE` das tabelas mestres:
```sql
CREATE POLICY "services_manager_write" ON public.services
  FOR ALL TO authenticated
  USING (
    tenant_id = current_tenant_id_from_auth_uid()
    AND current_is_tenant_manager_from_auth_uid(tenant_id)
  );
```

## Critérios de Aceite
- [ ] Barbeiro recebe erro 403 / RLS policy violation ao tentar mutação via REST em `services`
- [ ] Gerente / admin consegue alterar preços e cadastrar itens normalmente
- [ ] Teste automatizado de mutação REST validando negação para papel `barber`
--- FIM ISSUE 3 ---
````

ISSUE 4 — [Segurança] Arquivos de segredos locais desprotegidos contra commit no .gitignore (secrets.env.txt e *.bak_bom.txt)

````
--- ISSUE 4 ---
Título: [Segurança] Arquivos de segredos locais desprotegidos contra commit no .gitignore (secrets.env.txt e *.bak_bom.txt)
Labels: security, severidade-média

## Descrição do Problema
Existem arquivos locais na raiz do projeto contendo variáveis de ambiente e backups com chaves ativas (`secrets.env.txt` e
`env.local.bak_bom.txt`). A verificação com `git check-ignore` confirmou que o `.gitignore` atual não cobre essas extensões.

## Por que é explorável
Em fluxos de trabalho locais, desenvolvedores executam rotineiramente comandos como `git add .` ou `git add -A`. A ausência de
regras explícitas no `.gitignore` fatalmente levará à inclusão acidental desses arquivos no índice do Git.

## Evidência
```bash
git check-ignore secrets.env.txt env.local.bak_bom.txt
# Retorna código 1 (arquivos não estão ignorados pelo Git)
```

## Impacto
Risco iminente de novo vazamento de credenciais ativas para o histórico do repositório remoto.

## Sugestão de Correção
Incluir no arquivo `.gitignore`:
```gitignore
# Credenciais e backups locais
secrets.env.txt
*.bak_bom.txt
*.env.txt
```

## Critérios de Aceite
- [ ] `git check-ignore secrets.env.txt env.local.bak_bom.txt` retorna código 0
- [ ] Comandos `git status` não mostram arquivos de credenciais como untracked
--- FIM ISSUE 4 ---
````

ISSUE 5 — [Segurança] Alinhar permissões residuais de tabela com as policies de RLS (menor privilégio)

```
--- ISSUE 5 ---
Título: [Segurança] Alinhar permissões residuais de tabela com as policies de RLS (menor privilégio)
Labels: security, severidade-baixa

## Descrição do Problema
Identificadas concessões de permissões SQL a nível de tabela que excedem as políticas de RLS ativas:
1. `service_execution_participants`: GRANT SELECT, UPDATE, DELETE para o papel `anon` (`20260418100000:109`).
2. `feature_flags`: GRANT INSERT, UPDATE, DELETE para o papel `authenticated` (`20260807000000:80`), onde apenas `superadmin` tem política de RLS.

## Por que é explorável
A Row Level Security (RLS) protege as tabelas no estado atual porque o PostgreSQL bloqueia requisições sem policy correspondente. Contudo, manter grants excessivos quebra o princípio da defesa em profundidade: caso uma policy permissiva seja adicionada no futuro, o papel `anon` terá imediatamente permissão de tabela concedida.

## Evidência
- `supabase/migrations/20260418100000_add_service_execution_participants.sql:109`
- `supabase/migrations/20260807000000_phase_6_0_5_3_feature_flags.sql:80`

## Impacto
Débito técnico de segurança e falhas em ferramentas de auditoria estática de banco de dados.

## Sugestão de Correção
Criar migration de limpeza de privilégios:
```sql
REVOKE SELECT, UPDATE, DELETE ON public.service_execution_participants FROM anon;
REVOKE INSERT, UPDATE, DELETE ON public.feature_flags FROM authenticated;
```

Critérios de Aceite
- [ ] Inspeção de `information_schema.role_table_grants` sem privilégios excessivos
- [ ] Testes de regressão funcionais aprovados sem impactos em fluxos legítimos
--- FIM ISSUE 5 ---
```

ISSUE 6 — [Segurança] Refatorar fluxos de impressão para eliminar uso residual de document.write

```
--- ISSUE 6 ---
Título: [Segurança] Refatorar fluxos de impressão para eliminar uso residual de document.write
Labels: security, severidade-baixa

## Descrição do Problema
Os componentes de impressão `CashClosingPage.tsx` (linha 261) e `Comandas.tsx` (linha 1249) abrem janelas `about:blank` e injetam HTML via `printWindow.document.write()`.

## Por que é explorável
Ambos os fluxos já contam com tratamento preventivo através de `escapeHtml()` e anulação de contexto (`printWindow.opener = null`), neutralizando a vulnerabilidade de XSS. No entanto, o método `document.write` é desaconselhado pelos padrões modernos de desenvolvimento web e apresenta risco de regressão caso outros desenvolvedores introduzam novos dados dinâmicos sem a função de escape.

## Evidência
- `pages/CashClosingPage.tsx:261`
- `pages/Comandas.tsx:1249`

## Impacto
Risco potencial de regressão futura em manutenções no frontend.

## Sugestão de Correção
Adotar uma folha de estilos dedicada para impressão (@media print) ou componentes modais em tela com acionamento nativo de `window.print()`, evitando a criação de contextos `about:blank`.

## Critérios de Aceite
- [ ] Remoção de chamadas a `document.write()` no frontend
- [ ] Impressão de comanda e fechamento de caixa operando via estilos CSS de impressão nativos
--- FIM ISSUE 6 ---
```

Relatório emitido em conformidade com o protocolo SMG Security Control · 03/11/2026 · Formato A4